

Function Point Analysis & COCOMO Estimation

for

EVNet Sentinel: Anomaly Detection for EV Charging Infrastructure

Prepared by:

Group 1

Mukta Varak 24102A0013

Shruti Chaurasiya 24102A0018

Shardul Chogale 24102A0021

Neha Chavhan 24102A0022

Vidyalankar Institute of Technology

September 22, 2026

Contents

1	Function Point Analysis (FPA)	3
1.1	Reference: Data Flow Diagrams	3
1.1.1	Level 1 DFD Summary	3
1.1.2	Level 2 DFD Summary (Anomaly Detection — Process 4.0)	3
1.2	Weighting Factor Reference Table	4
1.3	Detailed Item Identification and Classification	5
1.3.1	User Inputs	5
1.3.2	User Outputs	5
1.3.3	User Inquiries (Queries to Data Stores)	5
1.3.4	Internal Logical Files (ILFs)	6
1.3.5	External Interface Files (EIFs) / API Calls	6
1.4	Unadjusted Function Point (UFP) Count Summary	7
2	Complexity Adjustment Factor (CAF)	8
2.1	Factor Scores for EVNet Sentinel	8
3	Function Point Calculation	10
3.1	Substituting Values	10
4	Mapping Function Points to Lines of Code	11
4.1	Language Gearing Factors	11
4.2	Estimated LOC Calculation	11
5	COCOMO Estimation — Basic Model	12
5.1	COCOMO Basic Model Formulae	12
5.2	COCOMO Coefficient Table	12
5.3	Project Classification: Organic	12
5.4	Calculation	12
5.4.1	Step 1: Effort (E)	12
5.4.2	Step 2: Development Time (D)	12
5.4.3	Step 3: Average Staff Size (SS)	13
6	Consolidated Summary	14
6.1	Interpretation	14

1 Function Point Analysis (FPA)

Function Point Analysis is a method for measuring the functional size of software based on what it *does* rather than how it is implemented. We derive the counts by systematically examining the Level 1 and Level 2 Data Flow Diagrams (DFDs) of EVNet Sentinel.

1.1 Reference: Data Flow Diagrams

1.1.1 Level 1 DFD Summary

The Level 1 DFD decomposes EVNet Sentinel into five major processes:

1. **Process 1.0 — Data Preprocessing:** Accepts raw CICEVSE2024 CSV datasets, cleans, masks PII, and performs feature engineering.
2. **Process 2.0 — Model Training:** Trains static classifiers (RF, SVM, LR, DT) and the online ARF+ADWIN model using processed data.
3. **Process 3.0 — Traffic Simulation:** Replays labelled traffic and injects synthetic attack patterns.
4. **Process 4.0 — Anomaly Detection:** Classifies incoming traffic against trained models and identifies anomalies.
5. **Process 5.0 — Alert and Monitoring:** Serves alerts, metrics, and dashboard data to the Security Analyst via REST/WebSocket.

Data stores identified: **D1** Dataset Storage, **D2** Processed Data, **D3** Trained Models, **D4** Anomaly Reports.

External entities: **Administrator/Researcher**, **Traffic Simulator**, **Security Analyst**.

1.1.2 Level 2 DFD Summary (Anomaly Detection — Process 4.0)

Process 4.0 is further decomposed into six sub-processes:

- **4.1** Receive Traffic
- **4.2** Prepare Features
- **4.3** Load ML Model
- **4.4** Classify Traffic
- **4.5** Identify Anomaly
- **4.6** Generate Detection Result

1.2 Weighting Factor Reference Table

The standard IFPUG weighting factors used for each domain characteristic are:

Domain Characteristic	Simple	Average	Complex
No. of User Inputs	3	4	6
No. of User Outputs	4	5	7
No. of User Inquiries	3	4	6
No. of Files (ILFs)	7	10	15
No. of External Interfaces	5	7	10

1.3 Detailed Item Identification and Classification

Each item is identified from the Level 1 and Level 2 DFDs, the SRS, and the codebase. Every item is individually classified as **Simple (S)**, **Average (A)**, or **Complex (C)** based on the number of data element types (DETs) and file types referenced (FTRs/RETs).

1.3.1 User Inputs

User inputs are data-entry or control transactions that cross the application boundary inward.

#	Input Item	DFD Reference	Complexity	Weight
I1	Raw Dataset Upload (CICEVSE2024 CSVs)	L1: Admin → P1.0	Average	4
I2	Training Request and Hyperparameters	L1: Admin → P2.0	Complex	6
I3	Simulation Configuration Parameters	L1: Traffic Sim → P3.0	Average	4
I4	Synthetic Attack Injection Patterns	L1: Traffic Sim → P3.0	Complex	6
I5	Model Selection for Prediction (model_name)	L2: 4.3 Load ML Model	Simple	3
I6	Network Traffic Feature Vector (/predict)	L2: 4.1 Receive Traffic	Complex	6
I7	Monitoring/Dashboard Request	L1: Analyst → P5.0	Simple	3
I8	Drift Detection Threshold Configuration	L1: Admin → P2.0	Average	4
Input Subtotal				36

Breakdown: 2 Simple $\times$ 3 = 6, 3 Average $\times$ 4 = 12, 3 Complex $\times$ 6 = 18. **Total = 36**

1.3.2 User Outputs

User outputs are data or control transactions that cross the application boundary outward to the user/external entity.

#	Output Item	DFD Reference	Complexity	Weight
O1	Classification Report (per-model metrics)	L1: P2.0 → Admin	Complex	7
O2	Prediction Result (prediction_class)	L2: 4.6 → Analyst	Simple	4
O3	Live Alert Stream (WebSocket /ws/alerts)	L1: P5.0 → Analyst	Complex	7
O4	Dashboard Metrics and Visualisation Data	L1: P5.0 → Analyst	Complex	7
O5	Anomaly Report (logged to D4)	L1: P5.0 → D4	Average	5
O6	Confusion Matrix Data	L1: P2.0 → Admin	Average	5
O7	Concept Drift Event Notification	L1: P4.0 → P5.0	Average	5
Output Subtotal				40

Breakdown: 1 Simple $\times$ 4 = 4, 3 Average $\times$ 5 = 15, 3 Complex $\times$ 7 = 21. **Total = 40**

1.3.3 User Inquiries (Queries to Data Stores)

An inquiry is a paired input/output where input causes immediate retrieval with no derived data.

#	Inquiry Item	DFD Reference	Complexity	Weight
Q1	Health Check (GET /health, GET /api/status)	L1: P5.0	Simple	3
Q2	List Loaded Models (GET /)	L1: P5.0	Simple	3
Q3	Retrieve Processed Feature Format from D2	L2: D2 → 4.2	Average	4
Q4	Load Trained Model from D3 Model Registry	L2: D3 → 4.3	Average	4
Q5	Query Anomaly Report History from D4	L1: D4 → P5.0	Average	4
Inquiry Subtotal				18

Breakdown: 2 Simple $\times$ 3 = 6, 3 Average $\times$ 4 = 12. **Total = 18**

1.3.4 Internal Logical Files (ILFs)

ILFs are user-identifiable groups of logically related data maintained within the application boundary.

#	File / Data Store	DFD Reference	Complexity	Weight
F1	D1 — Raw Dataset Storage (CICEVSE2024 CSVs)	L1: D1	Complex	15
F2	D2 — Processed/Scaled Feature Store	L1: D2	Average	10
F3	D3 — Trained Model Registry (.pkl files)	L1: D3	Complex	15
F4	D4 — Anomaly Reports / Alert Logs	L1: D4	Average	10
F5	Scaler Configuration (StandardScaler.pkl)	Code: API	Simple	7
F6	Feature Engineering Configuration / Column Mapping	Code: Preprocess	Simple	7
File Subtotal				64

Breakdown: 2 Simple $\times$ 7 = 14, 2 Average $\times$ 10 = 20, 2 Complex $\times$ 15 = 30. **Total = 64**

1.3.5 External Interface Files (EIFs) / API Calls

EIFs are data referenced by the application but maintained outside its boundary (external systems, APIs).

#	External Interface	DFD Reference	Complexity	Weight
E1	REST API — POST /predict	L2: 4.4 Classify	Complex	10
E2	REST API — GET /api/status	L1: P5.0	Simple	5
E3	REST API — GET /health	L1: P5.0	Simple	5
E4	WebSocket — /ws/alerts (bidirectional)	L1: P5.0 → FE	Complex	10
E5	Next.js ↔ FastAPI HTTP Interface	Architecture	Average	7
External Interface Subtotal				37

Breakdown: 2 Simple $\times$ 5 = 10, 1 Average $\times$ 7 = 7, 2 Complex $\times$ 10 = 20. **Total = 37**

1.4 Unadjusted Function Point (UFP) Count Summary

Parameter	Simple	$\times$ Wt	Average	$\times$ Wt	Complex	$\times$ Wt	Total
User Inputs	2	$\times 3 = 6$	3	$\times 4 = 12$	3	$\times 6 = 18$	36
User Outputs	1	$\times 4 = 4$	3	$\times 5 = 15$	3	$\times 7 = 21$	40
User Inquiries	2	$\times 3 = 6$	3	$\times 4 = 12$	0	$\times 6 = 0$	18
Files (ILFs)	2	$\times 7 = 14$	2	$\times 10 = 20$	2	$\times 15 = 30$	64
External Interfaces	2	$\times 5 = 10$	1	$\times 7 = 7$	2	$\times 10 = 20$	37
Unadjusted Function Point Count Total							195

Count Total (CT) = 195

2 Complexity Adjustment Factor (CAF)

The 14 General System Characteristics (GSCs) are rated on a scale of 0–5 for our project:

Scale	Meaning	Description
0	No influence	Factor has no relevance to the project
1	Incidental	Factor is present but has negligible effect
2	Moderate	Factor has some influence
3	Average	Factor has an average degree of influence
4	Significant	Factor significantly affects the system
5	Essential	Factor is absolutely essential

2.1 Factor Scores for EVNet Sentinel

i	Factor (f_i)	Score	Justification for EVNet Sentinel
1	Backup and Recovery	3	Model checkpoints (.pkl) and dataset versioning; no enterprise-grade backup but reproducibility logs exist.
2	Data Communications	4	REST API + WebSocket between backend and Next.js frontend; real-time alert streaming.
3	Distributed Processing	1	Single-machine deployment; backend and frontend are separate processes but on local-host.
4	Performance Critical	4	Near-real-time anomaly detection with sub-second inference latency targets (PR-1 in SRS).
5	Existing Operating Environment	3	Runs on standard student hardware (Python 3.10+, Node.js 20+); cross-platform support.
6	Online Data Entry	4	Live traffic simulation with feature vector submission via REST; admin training controls.
7	Input Transaction Over Multiple Screens	2	Dashboard has multiple pages (metrics, simulation, investigation) but inputs are mostly single-screen.
8	Master Files Updated Online	3	Model registry (D3) updated on training; anomaly reports (D4) updated on detection events.
9	Information Domain Values Complex	4	39 network-traffic features, multi-class labels, drift-detection metadata; complex domain.
10	Internal Processing Complex	5	Multiple ML algorithms (RF, SVM, LR, DT, ARF+ADWIN), feature engineering pipeline, concept drift adaptation.
11	Code Designed for Reuse	3	Modular Python packages (<code>src/data_prep</code> , <code>src/models</code> , <code>src/api</code>); reusable React components.
12	Conversion/Installation in Design	2	Docker-based deployment; <code>Makefile</code> automation; no complex data migration.
13	Multiple Installations	2	Primarily local-only, but Docker enables multi-environment deployment.
14	Application Designed for Change	4	Plugin-style model registry allows adding new classifiers; frontend component architecture supports extension.
Sum $\sum f_i$		44	

3 Function Point Calculation

The Adjusted Function Point (FP) is calculated using the standard formula:

$$FP = \text{Count Total} \times \left(0.65 + 0.01 \times \sum_{i=1}^{14} f_i \right)$$

3.1 Substituting Values

$$\text{Count Total (CT)} = 195 \tag{1}$$

$$\sum_{i=1}^{14} f_i = 3 + 4 + 1 + 4 + 3 + 4 + 2 + 3 + 4 + 5 + 3 + 2 + 2 + 4 = 44 \tag{2}$$

$$\text{Complexity Adjustment Factor (CAF)} = 0.65 + (0.01 \times 44) \tag{3}$$

$$= 0.65 + 0.44 \tag{4}$$

$$= 1.09 \tag{5}$$

$$\text{Function Points (FP)} = 195 \times 1.09 \tag{6}$$

$$= \boxed{212.55} \tag{7}$$

$$\text{Function Points (FP)} \approx 213$$

4 Mapping Function Points to Lines of Code

To apply COCOMO, we must first convert Function Points to Lines of Code (LOC). This conversion uses *language-dependent gearing factors* (also called “backfiring ratios”) published by QSM and other industry benchmarks.

4.1 Language Gearing Factors

EVNet Sentinel is a multi-language project:

Language	LOC per FP	Source
Python 3	35	QSM / Industry average for Python
JavaScript/TypeScript (React/Next.js)	47	QSM / Industry average for JS frameworks

4.2 Estimated LOC Calculation

Since the project has both a Python backend/ML engine ($\approx 60\%$ of functionality) and a TypeScript/React frontend ($\approx 40\%$ of functionality), we use a weighted average:

$$\text{LOC}_{\text{Python}} = 213 \times 0.60 \times 35 = 4,473 \text{ LOC} \quad (8)$$

$$\text{LOC}_{\text{TypeScript}} = 213 \times 0.40 \times 47 = 4,004 \text{ LOC} \quad (9)$$

$$\text{Total Estimated LOC} = 4,473 + 4,004 = \boxed{8,477 \text{ LOC}} \quad (10)$$

$$\text{KLOC} = \frac{8,477}{1,000} = \boxed{8.48 \text{ KLOC}}$$

5 COCOMO Estimation — Basic Model

The **Constructive Cost Model (COCOMO)** uses Lines of Code (KLOC) to estimate development effort, schedule, and team size.

5.1 COCOMO Basic Model Formulae

$$\text{Effort (E)} = a_b \times (\text{KLOC})^{b_b} \quad (\text{in Person-Months}) \quad (11)$$

$$\text{Development Time (D)} = c_b \times (E)^{d_b} \quad (\text{in Months}) \quad (12)$$

$$\text{Average Staff Size (SS)} = \frac{E}{D} \quad (\text{in Persons}) \quad (13)$$

5.2 COCOMO Coefficient Table

Software Project Type	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

5.3 Project Classification: Organic

EVNet Sentinel is classified as an **Organic** project because:

- The team is small (4 members) with familiarity in the application domain (ML, web development).
- The project size is moderate (< 50 KLOC).
- The system does not have rigid real-time or safety-critical constraints.
- It is an academic/research prototype, not an embedded or large-scale enterprise system.

5.4 Calculation

Using the **Organic** coefficients: $a_b = 2.4$, $b_b = 1.05$, $c_b = 2.5$, $d_b = 0.38$, and KLOC = 8.48:

5.4.1 Step 1: Effort (E)

$$E = a_b \times (\text{KLOC})^{b_b} \quad (14)$$

$$= 2.4 \times (8.48)^{1.05} \quad (15)$$

$$= 2.4 \times 9.21 \quad (16)$$

$$= \boxed{22.10 \text{ Person-Months}} \quad (17)$$

5.4.2 Step 2: Development Time (D)

$$D = c_b \times (E)^{d_b} \quad (18)$$

$$= 2.5 \times (22.10)^{0.38} \quad (19)$$

$$= 2.5 \times 3.22 \quad (20)$$

$$= \boxed{8.05 \text{ Months}} \quad (21)$$

5.4.3 Step 3: Average Staff Size (SS)

$$SS = \frac{E}{D} \tag{22}$$

$$= \frac{22.10}{8.05} \tag{23}$$

$$= \boxed{2.75 \text{ Persons} \approx 3 \text{ Persons}} \tag{24}$$

6 Consolidated Summary

Metric	Value
Unadjusted Function Point Count (CT)	195
Sum of Complexity Adjustment Factors ($\sum f_i$)	44
Complexity Adjustment Factor (CAF)	1.09
Adjusted Function Points (FP)	212.55 $\approx$ 213
Estimated LOC (Python component)	4,473 LOC
Estimated LOC (TypeScript component)	4,004 LOC
Total Estimated LOC	8,477 LOC
KLOC	8.48
COCOMO Model Type	Organic
Effort (E)	22.10 Person-Months
Development Time (D)	8.05 Months
Average Staff Size (SS)	$\approx$ 3 Persons

6.1 Interpretation and Gantt Chart Alignment

- The system requires approximately **22 Person-Months** of effort. With 4 team members working across a 4-month semester, this implies $22/4 = 5.5$ months of equivalent full-time effort per person — reasonable since students also carry coursework, and not all members are coding simultaneously on all phases.
- The COCOMO development time of $\approx$ **8 months** represents the theoretical *full lifecycle* duration (including requirements, design, verification, and documentation overhead). The project’s **Gantt chart spans 88 working days ($\approx$ 4 months)**, which is a compressed academic timeline. The factor-of-two difference is expected: COCOMO assumes sequential, industry-standard phase gates, while the Gantt chart leverages parallel task tracks (ML Engine, Simulation, and Dashboard run concurrently) and pre-existing tooling (scikit-learn, Next.js, River).
- The average staff size of $\approx$ **3 persons** is consistent with the actual team size of 4 members (the slight overestimate accounts for the fact that not all members work full-time on coding at every phase).
- The function point count of **213 FP** reflects a moderately complex system with significant data processing, multiple ML models, real-time communication (WebSocket), and a full-featured frontend dashboard.